

Uniprocessor scheduling

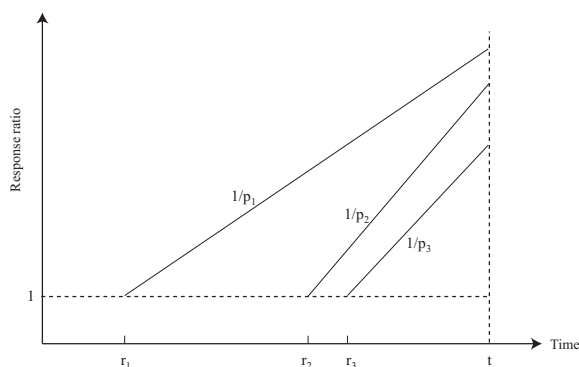
1 Een variant op highest response ratio next

Een variant van HRRN, minimax response ratio genaamd, heeft als doelstelling de maximale response ratio te minimaliseren. Bedenk een scheduling policy die hieraan voldoet. Inspireer je op het voorbeeld van Figuur 1; stel dat we een scheduling beslissing moeten maken op tijd t . In welke volgorde moeten taken t_1 , t_2 en t_3 worden uitgevoerd (vanaf tijd t), opdat de maximale response ratio $\max_s((w + p_s)/p_s)$ zo klein mogelijk is?

Solution: De response ratio wordt bepaald wanneer het proces is afgelopen. Kijk naar het tijdstip waarop alle processen zijn afgelopen (dit is $T = t + \sum_s p_s$). Kies het proces waarvoor $(T - r_s)/p_s$ het kleinst is als laatste proces om te schedulen. Doe hetzelfde om het voorlaatste proces te bepalen, enzovoort.

Bewijs optimaliteit:

- Noem P de taak met de hoogste response ratio, de tijd waarop deze taak eindigt noemen we T_P .
- Stel dat er een betere volgorde is, dan geldt ofwel:
 - P eindigt in deze betere volgorde later. Vermits de response ratio stijgt als een proces later wordt geplaatst, zou P een hogere response ratio hebben, en bijgevolg ook een hogere maximale response ratio. Dit kan niet.
 - P eindigt in deze betere volgorde eerder. Bijgevolg moet er minstens een andere job Q later geplaatst zijn, die eindigt op of na T_P . Maar we wisten al dat de response ratio van P op dit punt de kleinste was. Dit kan eveneens niet.



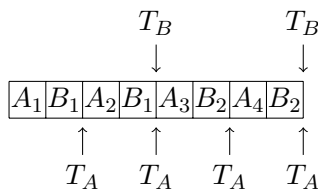
Figuur 1: Reponse ratio in functie van de tijd

2 Rate monotonic scheduling

Zoals reeds vermeld in de cursus, kan men aantonen dat RMS steeds de deadlines zal respecteren indien $U = \sum_{j=1}^n p_j/T_j \leq n(2^{1/n} - 1)$

1. Geef een voorbeeld van jobs (minstens 2 jobs) die met RMS hun deadlines halen, en U gelijk is aan 1.

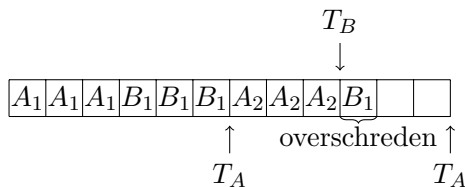
Solution: $T_A = 2, p_A = 1, T_B = 4, p_B = 2, U = \frac{1}{2} + \frac{2}{4} = 1$



2. Geef een voorbeeld van jobs die met RMS hun deadlines NIET halen, en U is hoogstens 0.95.

Solution:

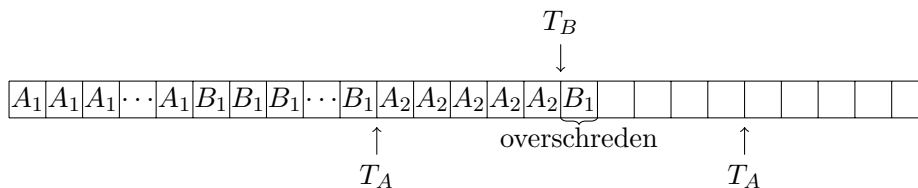
$T_A = 6, p_A = 3, T_B = 9, p_B = 4, U = \frac{3}{6} + \frac{4}{9} = 0.9444$



3. Geef een voorbeeld van jobs die met RMS hun deadlines NIET halen, en U is hoogstens 0.84.

Solution: Vorige oefening veralgemeend.

$T_A = 2n, p_A = n, T_B = 3n, p_B = n + 1, U = \frac{n}{2n} + \frac{n+1}{3n} = \frac{1}{2} + \frac{1}{3} + \frac{1}{3n}$



Geeft $U \geq 0.84$ vanaf $n \geq 50$.

Disk Management

1 Random seek

We beschouwen een harddisk met N tracks. Veronderstel nu dat de gevraagde sectoren random verdeeld zijn over alle tracks en onafhankelijk van elkaar. Bereken nu het gemiddeld aantal tracks dat de diskhead moet afleggen tussen twee requests, als volgt:

1. Wat is de kans dat de disk head op positie i staat en naar positie j moet?

Solution: $\frac{1}{N^2}$

2. Hoeveel tracks moeten er afgelegd worden van i naar j ?

Solution: $|i - j|$

3. Combineer 1 en 2 door alle mogelijke combinaties van i en j na te gaan, en werk uit.

Solution:

$$\sum_{i=1}^N i = \frac{1}{2}(n^2 + n)$$
$$\sum_{i=1}^N i^2 = \frac{1}{6}(2n^3 + 3n^2 + n)$$

$$\begin{aligned}
\sum_{i=1}^N \sum_{j=1}^N \frac{1}{N^2} |i-j| &= \frac{1}{N^2} \sum_{i=1}^N \left(\sum_{j=1}^i (i-j) + \sum_{j=i+1}^N (j-i) \right) \\
&= \frac{1}{N^2} \sum_{i=1}^N \left(i^2 - \sum_{j=1}^i j - (N-i)i + \sum_{j=i+1}^N j \right) \\
&= \frac{1}{N^2} \sum_{i=1}^N \left(i^2 - \frac{1}{2}(i^2 + i) - (N-i)i + \frac{1}{2}(N^2 + N) - \frac{1}{2}(i^2 + i) \right) \\
&= \frac{1}{N^2} \sum_{i=1}^N \left(i^2 - (N+1)i + \frac{1}{2}(N^2 + N) \right) \\
&= \frac{1}{N^2} \left(\frac{1}{6}(2N^3 + 3N^2 + N) - (N+1)\frac{1}{2}(N^2 + N) + \frac{N}{2}(N^2 + N) \right) \\
&= \frac{1}{N^2} \left(\frac{1}{3}N^3 - \frac{1}{3}N \right) = \frac{N}{3} - \frac{1}{3N}.
\end{aligned}$$

4. Gegeven dat N groot is, geef een benadering voor het aantal af te leggen tracks, relatief t.o.v. het totaal aantal tracks.

Solution:

$$\lim_{N \rightarrow \infty} \frac{\frac{N}{3} - \frac{1}{3N}}{N} = \lim_{N \rightarrow \infty} \frac{N^2 - 1}{3N^2} = \frac{1}{3}$$

2 Scheduling algoritmes

Veronderstel dat we een IO controller hebben die het SSTF algoritme hanteert, en dat de diskhead op de middenste track van de disk staat.

1. Geef nu een reeks van 8 requests op, waarbij de disk head tussen elke request over het midden van de disk moet langskomen. Geef de minimale (in de zin van totaal af te leggen tracks) oplossing voor dit probleem. (Je mag de requests noteren relatief ten opzichte van de middenste track; met andere woorden als de middenste track 512 is, mag je track 511 aangeven als -1 .)

Solution: We starten (arbitrair) met de disk head richting positieve tracknummers (dus een request voor track 1). Merk op dat de batch niet per request, maar in zijn geheel wordt bekeken door SSTF. De volgorde waarin de requests opgelijst worden

Nu zorgen we er steeds voor dat de afstand naar de volgende track aan dezelfde kant (t.o.v. het midden) net 1 groter is dan de afstand naar de andere kant. Bv. de afstand van 1 naar -2 is 3, terwijl de afstand van 1 naar 5 gelijk is aan 4 ($= 3+1$). We herhalen dat tot we 8 requests hebben en krijgen de batch:

$$0 \rightarrow 85 \rightarrow -170 \rightarrow 85 + 255 - 340$$

[illegible][illegible]

C) Raid 0+5

Raid 0

Raid 5

D) Raid 5+0

Raid 5:

Raid 0

E) Raid 5+0

Raid 5:

Raid 0

F) Raid 5+0

Raid 5:

Raid 0

X 240GB

Y 240GB

A 400GB

B 360GB

C 320GB

D 320GB

E 360GB

F 400GB

4 RAID: robuust?

Gegeven dat je 12 harddisks hebt, ga na voor volgende RAID configuraties, hoeveel disks er maximaal mogen uitvallen, zonder dat het systeem faalt (niet noodzakelijk onherstelbaar). Berekenen hierna de kans dat het systeem operationeel blijft, wanneer twee hard disks falen.

- RAID 0+1
- RAID 1+0
- RAID 0+5 (alle 3 mogelijke opsplitsingen)
- RAID 5+0 (alle 3 mogelijke opsplitsingen)

Solution: Bij het maximum aantal disks dat kan falen, maakt het verschil welke disks falen. Bij alle systemen kan er slechts 1 zeker uitvallen; een “verkeerd gekozen” tweede disk doet het systeem al stop. Bij “goed gekozen” disks, mogen er soms tot 6 disk uitvallen.

Om de kans te bepalen dat het systeem operationeel blijft, wanneer twee hard disks falen, kunnen we eerst een willekeurige disk uitkiezen die al faalt. Nu tellen we hoeveel van de overbleven disks niet mogen uitvallen.

	Max. aantal disks dat kan falen	Kans operationeel bij twee disks
X	1 – 6	5/11
Y	1 – 6	10/11
A	1 – 2	1/11
B	1 – 3	2/11
C	1 – 4	3/11
D	1 – 4	9/11
E	1 – 3	8/11
F	1 – 2	6/11